

Campo Minado

Campo minado é um jogo de estratégia muito popular e que possui diversas versões disponíveis para o público. Essencialmente, o jogo consiste em um tabuleiro de n x m (linhas e colunas) e k minas que estão escondidas em posições aleatórias do tabuleiro, sendo que uma determinada posição só pode armazenar no máximo uma mina. O objetivo do jogador é descobrir a localização das minas no tabuleiro.

No presente problema, sua missão será um pouco diferente. Você deverá desenvolver um programa que recebe como entrada o tamanho do tabuleiro do campo minado, representado por dois valores inteiros n e m. Logo em seguida, o programa receberá n linhas de texto representando a configuração inicial do tabuleiro, onde o caractere ***** (asterisco) indica a presença de uma mina e o caractere **.** (ponto) indica uma posição não revelada. No exemplo abaixo, temos um tabuleiro com dimensão 4 x 4 e sua distribuição inicial contendo duas minas.

```
*...
....
.*..
....
```

Sua tarefa é, a partir da matriz de entrada, preencher as posições contendo **.** (ponto) com o respectivo número inteiro que indica a quantidade de minas existentes nas oito posições adjacentes a ele. Para ilustrar, observe abaixo a saída esperada, considerando o exemplo apresentado acima. O valor 0 (zero) indica a inexistência de mina nas oito posições adjacentes. O valor 1 (um) indica a presença de uma mina dentre as oito posições, o valor 2 (dois) indica a presença de duas minas dentre as oito posições adjacentes e, assim, sucessivamente.



```
*100
2210
1*10
1110
```

ENTRADA

A primeira linha contém dois valores inteiros n x m (0 < n,m 100) que representam o número de linhas e colunas do tabuleiro, respectivamente. Cada uma das próximas n linhas seguintes contém exatamente m caracteres que representam as colunas do tabuleiro. O caractere **.** (ponto) indica posições não reveladas e o caractere ***** (asterisco) indica a presença de uma mina.

SAÍDA

A saída deve conter n linhas onde cada posição que contém o caractere **.** (ponto) deve ser substituída pelo número de minas existentes em suas oito posições adjacentes.

CRÉDITOS

Adaptado da versão apresentada em SKIENA, S. S.; REVILLA, M. A. Programming Challenges. [s.l.] Springer Science & Business Media, 2006.

CharCode 2025

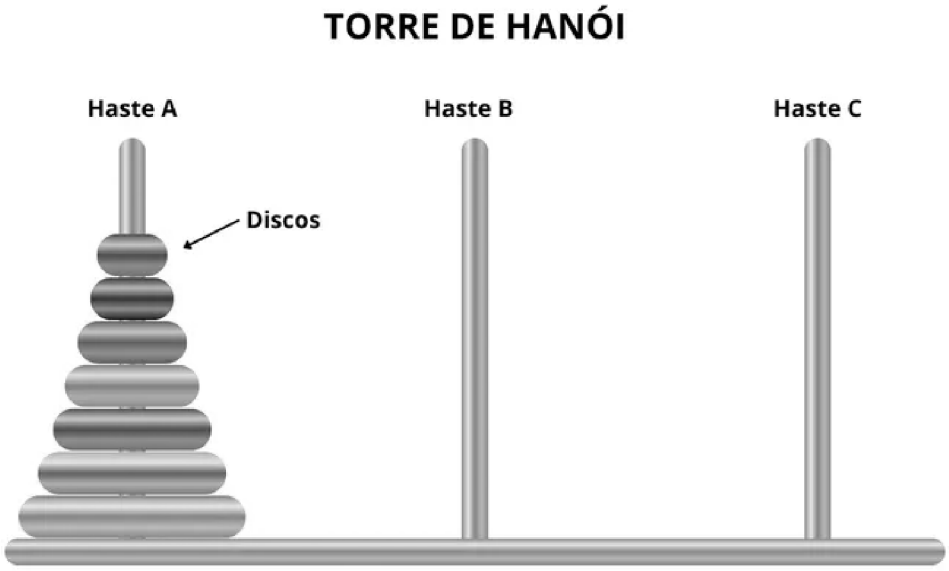
Maratona de Programação



Input samples	Output samples
4 4 *... .... .*..	*100 2210 1*10 1110
3 5 **... ..... .*...	**100 33200 1*100
6 10 *......... .*.*...... .... *........* .... *.........	*221100000 2*2*100000 2221100011 *10000001* 2200000011 *100000000
10 10 *......... .*.*...... .... *........* .... *....**... *......... **........ *......... *.........	*221100000 2*2*100000 2221100011 *10000001* 2200122111 *2001**100 *410122100 **10000000 *410000000 *200000000

Torre de Hanói

Torre de Hanói é um famoso quebra-cabeça/problema matemático criado pelo matemático francês Édouard Lucas. Ele consiste em uma base contendo três hastes (A, B e C) e um conjunto de n discos com diâmetros diferentes. O início do jogo se dá posicionando todos os discos na haste A, uns sobre os outros, em ordem crescente de diâmetro, de cima para baixo. A imagem abaixo demonstra a configuração inicial do quebra-cabeça, considerando 7 discos.



O objetivo do jogo é levar todos os discos da haste A para a haste C, utilizando a haste B como auxiliar e cumprindo as seguintes regras:

- Apenas um disco pode ser movido por vez.
- Um disco só pode ser movido se ele estiver no topo da pilha de sua haste.
- Um disco maior nunca pode ficar em cima de um disco menor.



Seu desafio é construir um programa que descubra a menor quantidade correta de movimentos para mover todos os n discos da haste para A haste C. Para saber qual a menor quantidade de movimentos, basta calcular $2^n - 1$, ou seja, para n = 3 (3 discos) a menor quantidade correta de movimentos é 7 ($2^3 - 1 = 7$).

ENTRADA

Um valor inteiro n (n >= 1 && n <= 64) que indica o número de discos que serão utilizados no jogo.

SAÍDA

A menor quantidade correta de movimentos para mover todos os n discos da haste para A haste C, sendo apresentado um movimento por linha.



Input samples	Output samples
2	Move disco 1 de A para B Move disco 2 de A para C Move disco 1 de B para C
3	Move disco 1 de A para C Move disco 2 de A para B Move disco 1 de C para B Move disco 3 de A para C Move disco 1 de B para A Move disco 2 de B para C Move disco 1 de A para C
4	Move disco 1 de A para B Move disco 2 de A para C Move disco 1 de B para C Move disco 3 de A para B Move disco 1 de C para A Move disco 2 de C para B Move disco 1 de A para B Move disco 4 de A para C Move disco 1 de B para C Move disco 2 de B para A Move disco 1 de C para A Move disco 3 de B para C Move disco 1 de A para B Move disco 2 de A para C Move disco 1 de B para C



Contrapeso

A LogiSafe é uma transportadora que trabalha com cargas sensíveis e precisa garantir estabilidade durante o deslocamento. Para cada viagem, os operadores montam compartimentos com exatamente duas caixas (um par) que servirão como contrapesos. A fim de evitar balanços e garantir segurança, a soma dos pesos das duas caixas de cada par deve ser exatamente igual à capacidade ideal do contrapeso C.

Atualmente a conferência é feita manualmente: os operadores inspecionam listas e tentam combinar caixas, o que consome tempo e é sujeito a erros quando há muitas caixas no depósito. Para automatizar e acelerar o processo, a LogiSafe contratou sua equipe para desenvolver um programa que, dado o inventário de caixas e o valor C, determine quantos pares distintos de caixas podem formar um contrapeso perfeito. O resultado ajudará a planejar as cargas rapidamente e sem retrabalho.

Importante: um dos estagiários tentou implementar a solução verificando todas as combinações possíveis de duas caixas. Isso até funciona em pequenos depósitos, mas o procedimento ficou impraticável em horários de pico, quando o sistema precisa analisar muitas caixas rapidamente. Por isso, sua solução precisa ser eficiente.

Tarefa

Dada a lista de pesos das N caixas, calcule quantos pares distintos (i, j) com $0 \le i < j < N$ satisfazem $V_i + V_j = C$.

Entrada

A primeira linha contém dois inteiros N e C ($2 \le N \le 10^5$, $2 \le C \le 2000$).
A segunda linha contém N inteiros $V_1, V_2, ..., V_N$ ($1 \le V_i \le 1000$) — pesos das caixas, na ordem de registro.

Saída

Imprima um único inteiro: a quantidade de pares distintos (i, j) com i < j tal que $V_i + V_j = C$.

Input samples	Output samples
5 10 3 4 5 6 7	2
6 8 2 3 6 5 4 6	3
4 15 7 5 8 10	2



Resgate Subterrâneo

Um robô explorador sofreu uma falha crítica nos sensores e ficou "cego" durante uma operação de resgate numa instalação subterrânea instável. O piso da instalação está contaminado: qualquer movimento fora de um único corredor seguro destrói imediatamente o robô. Um operador remoto, que conhece o mapa, deve transmitir ao robô a sequência de instruções que descreve o único caminho seguro até a saída. O robô deve seguir estritamente essas instruções.

Você deve escrever um programa que, dado a posição e orientação iniciais do robô e a sequência de instruções enviada pelo operador, simule a caminhada e imprima a sequência completa de posições visitadas. A solução será aceita se o conjunto de posições produzidas coincidir exatamente com o corredor seguro.

Entrada

A entrada contém um caso de teste com o formato abaixo:

- Linha 1: X_0 Y_0
  Dois inteiros ($0 \le X_0 \le 99$, $0 \le Y_0 \le 99$), separados por um espaço — posição inicial do robô. Aqui (X, Y) denota coordenadas cartesianas no mapa.
- Linha 2: D
  Um caractere entre 'N', 'S', 'L', 'O' — orientação inicial do robô:
  N = Norte, S = Sul, L = Leste, O = Oeste.
- Linha 3: N
  Um inteiro ($1 \le N \le 100$) — número de instruções.
- Linha 4: sequência de N instruções separadas por espaço.
  Cada instrução é um dos caracteres:
  F - Andar para Frente (mover 1 célula na direção atual)



E - Virar à Esquerda (girar 90º à esquerda, então mover 1 célula)
D - Virar à Direita (girar 90º à direita, então mover 1 célula).

Regras das instruções:

- Cada instrução resulta em exatamente um movimento de uma célula após (se aplicável) ajustar a orientação.
- As instruções produzidas pelo operador de resgate nunca levarão o robô para fora dos limites $0 \leq X \leq 99$ e $0 \leq Y \leq 99$.
- Convenção de sinais:
  X cresce de Oeste para Leste
  Y cresce de Norte para Sul.

Saída

Imprima exatamente N+1 linhas. A linha i ($0 \leq i \leq N$) deve conter dois inteiros X_i Y_i, separados por espaço, representando a posição do robô após i instruções aplicadas:

- Linha 0: X_0 Y_0 (posição inicial)
- Linha i ($1 \leq i \leq N$): posição (X_i $Y_{i)}$ após aplicar a i-ésima instrução.



Input samples	Output samples
5 5 N 6 F F E F D E	5 5 5 4 5 3 4 3 3 3 3 2 2 2
10 15 O 10 F E D F F D D E F D	10 15 9 15 9 16 8 16 7 16 6 16 6 15 7 15 7 14 7 13 8 13
0 0 L 3 F D E	0 0 1 0 1 1 2 1

Frequência das Notas Musicais

As notas musicais podem ser identificadas por letras para facilitar a escrita, aumentar a velocidade de leitura e facilitar a comunicação com músicos de outros países. A definição das letras e suas notas correspondentes é a seguinte:

Nota	Universal		Nota	Universal		Nota	Universal
Dó	C		Mi	E		Sol#	G#
Dó#	C#		Fá	F		Lá	A
Ré	D		Fá#	F#		Lá	A#
Ré#	D#		Sol	G		Si	B

Observe que a Tabela abaixo forma toda a escala cromática de uma ***oitava***. A nota Lá (A), por exemplo, corresponde a uma frequência de 440 Hz. Quando uma frequência é multiplicada por 2, a nota permanece a mesma. Assim, as frequências de 220Hz e 880 Hz são também notas Lá, só que uma oitava abaixo e uma oitava acima, respectivamente. Nota musical Lá (A) em diferentes frequências – quanto mais baixa a frequência, mais grave o som.

Nota musical Lá (A) em diferentes frequências – quanto mais baixa a frequência, mais grave o som							
Oitava - *oitava*	0	1	2	3	4	5	6
Frequência (Hz) - $Freq_{oitava}$	55	110	220	440	880	1760	3520

Para cada frequência, temos um som diferente. Todas as notas devem ter a mesma distância umas das outras. Essa distância é chamada de ***semitom***. Por meio da análise de frequências, descobriu-se que multiplicando a frequência de uma nota musical por 1.0594, chegava-se à nota seguinte. Depois de multiplicar a frequência da nota Lá pelo



número 1.0594 doze vezes, retornamos à nota Lá. Note que a raiz duodécima de 2 – $2^{1/12}$ – é 1.059463094.

	Semitom / Nota	Frequência (Hz)	Multiplicador
OITAVA 3	A	440	$2^{0/12}$
	A#	466.16	$2^{1/12}$
	B	493.88	$2^{2/12}$
	C	523.25	$2^{3/12}$
	C#	554.37	$2^{4/12}$
	D	587.33	$2^{5/12}$
	D#	622.25	$2^{6/12}$
	E	659.26	$2^{7/12}$
	F	698.46	$2^{8/12}$
	F#	739.99	$2^{9/12}$
	G	783.99	$2^{10/12}$
	G#	830.61	$2^{11/12}$
	A	880	$2^{0/12}$

Um afinador de instrumentos de corda lê a frequência da vibração da corda ($Freq_{corda}$) e indica como está a afinação em uma das notas musicais para o músico saber se tem que apertar ou afrouxar a corda para afinar o instrumento musical. A partir do código universal da nota musical e da oitava, é possível calcular a frequência da vibração da corda.

Entrada

A entrada são duas linhas, onde a primeira linha é a oitava ***O*** (0 <= ***O*** <= 9) indicando a frequência de base da oitava e a segunda linha é uma string com o código universal de



determinada nota musical

Saída

A saída é um número em ponto flutuante com duas casas depois da vírgula indicando a frequência da vibração da corda em Hz.

Input samples	Output samples
0 A	55.00
1 A#	116.54
2 B	246.94
7 E	10548.08
8 F	22350.61
9 F#	47359.29



Uma batalha pela vitória

Na arena da GladCode, diversos gladiadores entram em combate buscando a gloriosa vitória com base em suas habilidades e, também, através de uma habilidade especial que poucos deles conhecem, o uso de anéis especiais. Esses gladiadores possuem as seguintes características:

Atributo	Tipo de dado
Nome	String sem espaços
Pontos de força	Inteiro positivo
Pontos de agilidade	Inteiro positivo
Cor da pedra do anel	Caractere

Quando dois gladiadores se enfrentam, o vencedor é determinado por um critério bem simples, o que tiver maior poder vence. Para o cálculo desse poder, observe o seguinte:

1 – O poder básico de cada gladiador é calculado conforme a fórmula abaixo.

Poder = força * 2 + agilidade

2 – Esse poder será acrescido conforme o uso de anéis com pedras de cores diferentes (todo gladiador possui um anel).

Cor do Anel	Bônus (%)
V (Vermelho)	+50%
A (Azul)	+40%
R (Roxo)	+30%
B (Branco)	+20%
P (Preto)	+10%

No duelo, o gladiador com maior poder vence, contudo, em casos de empate, considere as regras a seguir:

- Em caso de empate, vence o gladiador que tiver maior agilidade.
- Se ainda der empate, vence o primeiro gladiador listado no par de combate.

As batalhas ocorrem entre pares. O vencedor da primeira luta irá batalhar com o próximo gladiador na lista, e assim sucessivamente até que a lista tenha finalizado.

Entrada

- A primeira linha contém um inteiro G (2 ≤ G ≤ 1000) — o número de gladiadores.
- As próximas G linhas descrevem cada gladiador no formato:
- nome força agilidade cor_do_anel

- nome: string sem espaços
- força, agilidade: inteiros (1 ≤ força, agilidade ≤ 1000)
- cor do anel: caractere representando a cor, por exemplo "V", "A", "R", "B" e "P".

Saída

- Para cada batalha, exibir o nome do gladiador vencedor
- Ao final exibir a mensagem "Glorioso vencedor: <nome_do_gladiador>"



Input samples	Output samples
5 Maximus 200 150 B Minimus 180 200 A Commodus 190 160 P Vanius 170 210 V Titus 160 180 R	Minimus Minimus Vanius Vanius Glorioso vencedor: Vanius
10 Maximus 119 175 B Minimus 220 173 B Spartacus 135 179 A Vanius 147 268 V Commodus 272 291 B Spiculus 120 227 A Vinicius 267 255 V Crixus 267 255 A Cassius 228 115 V Atticus 243 266 B	Minimus Minimus Vanius Commodus Commodus Vinicius Vinicius Vinicius Vinicius Glorioso vencedor: Vinicius

Peças de cerâmica

Em um ateliê de cerâmica, cada peça é moldada a partir de um volume aproximado de argila, que depende de suas dimensões.

Rodolfo, por ser um ceramista iniciante, muitas vezes têm dificuldade em estimar quanta massa será necessária para produzir uma tigela, caneca ou prato, especialmente antes da queima, quando a peça ainda encolherá com a secagem.

Para facilitar o trabalho do Rodolfo, você deve criar um programa que calcule a quantidade aproximada de argila (em gramas) necessária para uma determinada peça, com base em suas dimensões e na densidade média da argila.

O volume da peça é estimado por uma forma geométrica simples (cilindro), usando a fórmula:

$V = pi * r^2 * h$

onde:

- r é o raio da peça (em cm);
- h é a altura (em cm);
- e o volume V deverá ser convertido para gramas usando a densidade média da argila (2,0 g/cm³), ou seja, cada 1 cm³ de volume de argila pesa 2 gramas.

Considere o valor de pi = 3,1415. Seu programa deve calcular o peso aproximado da argila, considerando ainda 10% a mais para compensar perdas no processo.



Por fim, o valor deve ser sempre arredondado para cima até o próximo número terminado em 0 ou 5, garantindo que o ceramista sempre tenha material suficiente.

Entrada

A entrada contém dois valores reais, separados por espaço:

- o diâmetro da peça (em cm);
- e a altura da peça (em cm).

Saída

Um número inteiro, representando a quantidade aproximada de argila em gramas, arredondado para o próximo número terminado em 0 ou 5.

Input samples	Output samples
2 2	15
8 8	885
10 6	1040
12 8	1995
25 12	12960

Feira de Ciências

Na Feira de Ciências do campus, estudantes de diferentes cursos apresentam trabalhos que misturam ciência, tecnologia, arte e reflexão social. Há grupos que exploram robótica e programação, outros que demonstram experimentos de química e física, e aqueles que propõem pesquisas de literatura, história, filosofia e expressão corporal.

Para organizar todas esses trabalhos no site oficial, a comissão precisa gerar um identificador único para cada um deles, formado pelas iniciais das palavras do título seguidas do ano do evento. O problema é que, com tantos nomes criativos e interdisciplinaridade, alguns títulos acabam gerando o mesmo identificador.

Seu desafio é criar um programa que analise todos os títulos e indique quantas oficinas geraram identificadores repetidos, ou seja, quantos estão fora do padrão e precisam ser renomeados antes da publicação no site.

Entrada

A primeira linha contém dois inteiros:

· N — o número de oficinas cadastradas ($1 \leq N \leq 100$);

· A — o ano do evento ($1 \leq A \leq 9999$).

As próximas N linhas contêm os títulos dos trabalhos, compostos apenas por letras minúsculas e espaços, com pelo menos uma palavra em cada título.

Saída



Um único número inteiro: a quantidade de trabalhos fora do padrão, ou seja, aqueles que geraram identificadores duplicados.

Input samples	Output samples
7 2025 robotica para iniciantes realidade aumentada basica redes de computadores relatos de vida e memoria roteiros de leitura critica robotica pratica introdutoria realidade aumentada basica	2
6 2025 historia e cinema historia e memoria poesia visual e som programacao criativa pintura com luz poesia visual e som	1
6 2025 projeto de biologia marinha contacao de historias infantis matematica e arte astronomia para iniciantes historia e filosofia da ciencia design e sustentabilidade	0



Gincana do IF

Durante a gincana do IFSUL, foi lançada uma tarefa solicitando às equipes que coletem e entreguem tampinhas plásticas de garrafas PET de cinco cores diferentes: azul, preta, verde, laranja ou roxa.

A equipe deverá receber **X** pontos por cada tampinha entregue. Além disso, como bônus, cada kit completo, contendo uma tampinha de cada cor, vale **Y** pontos extras.

Para ajudar a Comissão Organizadora, você deverá criar um programa que calcule a pontuação total de uma equipe, levando em consideração:

- A pontuação X para cada tampinha;
- O bônus Y para cada kit completo;
- As quantidades de tampinhas entregues pela equipe, para cada cor.

Entrada

A entrada consiste em duas linhas:

```
X Y
A P V L R
```

Onde:

- X = pontos por tampinha
- Y = pontos por kit completo
- A = quantidade de tampinhas azuis
- P = quantidade de tampinhas pretas
- V = quantidade de tampinhas verdes



- L = quantidade de tampinhas laranjas
- R = quantidade de tampinhas roxas

Limites:

- $1 \leq X, Y \leq 1000$
- $0 \leq A, P, V, L, R \leq 10^6$

Saída

Um único número inteiro: a pontuação total da equipe.

Input samples	Output samples
1 3 53 67 59 82 74	494
2 5 88 14 77 38 91	686
1 5 77 0 94 8 16	195
3 2 61 73 59 88 67	1162



Mocitec

Devido ao enorme sucesso da MOCITEC e ao grande número de trabalhos inscritos, os espaços tradicionalmente utilizados (salas de aula e área em frente à cantina) não serão suficientes para abrigar todos os projetos. Diante disso, a Comissão Organizadora decidiu usar a passarela coberta que vai do Bloco 17 até a quadra para instalar novos estandes.

Após uma inspeção técnica no percurso, foram identificados **N pontos** que possuem condições adequadas para receber um estande (proximidade de tomadas, boa iluminação, circulação adequada, etc.). Cada ponto foi identificado por um número inteiro **X**, que representa a sua distância (em metros) a partir do início do trajeto, em frente ao Bloco 17

Agora, a comissão precisa escolher **K pontos** entre os **N possíveis** para alocar os novos estandes. Para garantir uma boa distribuição ao longo do percurso, evitando que fiquem muito próximos, melhorando a circulação e o conforto acústico, os pontos devem ser escolhidos de modo que **a menor distância entre quaisquer dois estandes seja a maior possível**.

Contudo, dado o tamanho do estande, **os pontos escolhidos devem estar a pelo menos 2 metros de distância um do outro**. Sua tarefa é escrever um programa que determine qual é a maior distância mínima possível entre dois estandes, respeitando o afastamento de 2 metros.

Entrada

A entrada consiste em duas linhas:



- A primeira linha contém dois valores inteiros, N e K;
- A segunda linha contém N valores inteiros, X_1, X_2, …, X_n, representando as posições possíveis para instalação dos estandes.

```
N K
X1 X2 … Xn
```

onde:

- **N** → quantidade de pontos possíveis para instalar estandes ($1 \leq N \leq 10^6$);
- **K** → quantidade de estandes que devem ser instalados ($K \geq 2$);
- **X_i** → posição de um ponto disponível para instalação ($0 \leq X_i \leq 10^6$).

Observação: os valores **X_i** podem ser fornecidos em qualquer ordem!

Saída

O programa deve imprimir **um único inteiro** representando a **maior distância mínima possível entre dois estandes**, respeitando o afastamento de 2 metros. Caso **não haja uma solução possível** (quando o número de estandes K for maior que o número de pontos N, ou quando não for possível manter 2 metros de distância mínima), o programa deve imprimir **0**.

CharCode 2025

Maratona de Programação



Input samples	Output samples
10 5 10 20 30 40 50 60 70 80 90 100	20
8 3 0 1 2 20 21 22 48 50	22
5 6 5 2 10 20 28	0
10 6 1 2 3 4 5 6 7 8 9 10	0
6 5 24 16 11 5 3 0	5